Summarize

Chapter 9. Testing

The world of automated testing has advanced significantly since I first started writing code, and every month there seems to be some new tool or technique to make it even better. But challenges remain regarding how to effectively and efficiently test our code's functionality when it spans a distributed system. This chapter breaks down the problems associated with testing finer-grained systems and presents some solutions to help you make sure you can release your new functionality with confidence.

Testing covers a lot of ground. Even when we are talking *only* about automated tests, there are a large number to consider. With microservices, we have added another level of complexity. Understanding what different types of tests we can run is important to help us balance the sometimes-opposing forces of getting our software into production as quickly as possible versus making sure our software is of sufficient quality. Given the scope of testing as a whole, I am not going to attempt a broad exploration of the topic. Instead, this chapter is primarily focused on looking at how testing of a microservice architecture is different when compared to less-distributed systems such as single-process monolithic applications.

Where testing is done has also shifted since the first edition of this book. Previously, testing was predominantly carried out before the software got to production. Increasingly, though, we look at testing our applications once they arrive in production—further blurring the lines between development and production-related activities; this is something we'll explore in this chapter before exploring testing in production more fully in Chapter 10.

Types of Tests

Like many consultants, I'm guilty of occasionally using quadrants as a way of categorizing the world, and I was starting to worry this book wouldn't have one. Luckily, Brian Marick came up with a fantastic categorization system for tests that fits right in.

Figure 9-1 shows a variation of Marick's quadrant from Lisa Crispin and Janet Gregory's book *Agile Testing*[1] that helps categorize the different types of tests.

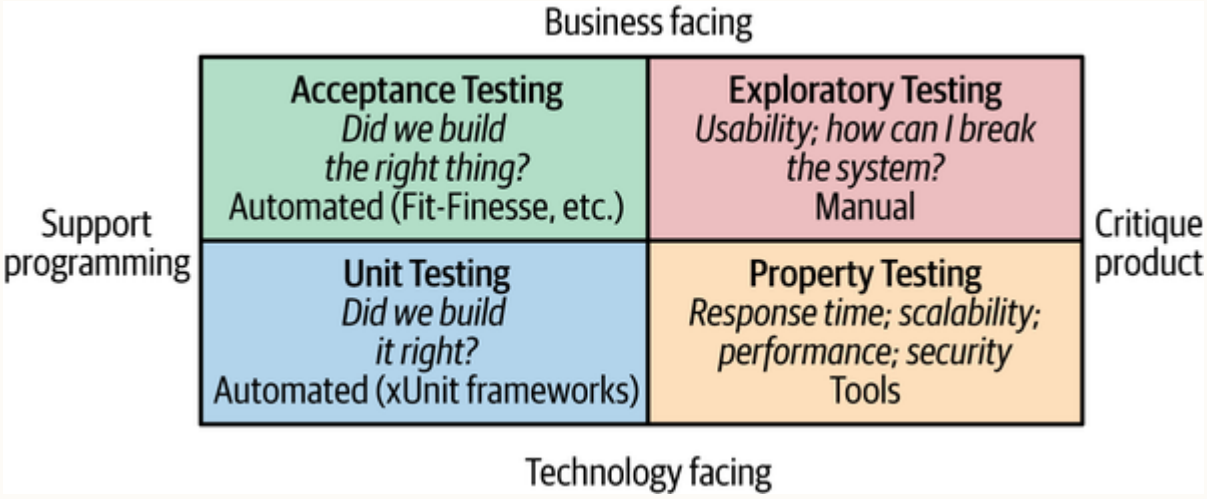


Figure 9-1. Brian Marick's testing quadrant. Lisa Crispin and Janet Gregory, *Agile Testing: A Practical Guide for Testers and Agile Teams*, © 2009

At the bottom of the quadrant, we have tests that are *technology facing*—that is, tests that aid the developers in creating the system in the first place. Property tests such as performance tests and small-scoped unit tests fall into this category; all are typically automated. The top half of the quadrant includes those tests that help the nontechnical stakeholders understand how your system works, which we call *business-facing* tests. These could be large-scoped, end-to-end tests, as shown in the Acceptance Testing quadrant at top left; or manual testing (as typified by user testing done against a UAT system), as shown in the Exploratory Testing quadrant.

At this point it's worth calling out the fact that the vast majority of these tests focus on *preproduction validation*. Specifically, we are using these tests to ensure that the software is of sufficient quality before it is deployed into a production environment. Typically, these tests passing (or failing) would be a gating condition for deciding whether the software should be deployed.

Increasingly, we are seeing the value of testing our software once we actually get into a production environment. We'll talk more about the balance between these two ideas later in the chapter, but for now it's worth highlighting a limitation of Marick's quadrant in this regard.

Each type of test shown in this quadrant has a place. Exactly how much of each test you want to do will depend on the nature of your system, but the key point to understand is that you have multiple choices in terms of how to test your system. The trend recently has been away from any large-scale manual testing in favor of automating as much repetitive testing as possible, and I certainly agree with this approach. If you currently

carry out large amounts of manual testing, I would suggest you address that before proceeding too far down the path of microservices, as you won't get many of their benefits if you are unable to validate your software quickly and efficiently.

MANUAL EXPLORATORY TESTING

In general, the shift from a monolithic architecture to a microservice architecture will have minimal impact on exploratory testing, aside from any wider organizational shift that might occur. As we'll explore in "Toward Stream-Aligned Teams", we'd expect the user interface for the application to also be broken down along team lines. In such a context, the ownership of manual testing may well change.

The ability to automate some tasks, such as verifying how something looks, was previously limited exclusively to manual exploratory testing. The maturing of tooling that enables visual assertions has allowed us to start automating tasks that were previously done manually. You shouldn't see this as a reason to have no manual testing, though; you should instead see it as a chance to free up testers' time to focus on less repetitive, *exploratory* testing.

When done well, manual exploratory testing is to a great extent about discovery. Setting aside time to explore the application as an end user can unearth issues that otherwise wouldn't be evident. Manual testing can also be vital in situations in which an automated test is infeasible to implement, perhaps because of the cost of writing the test. Automation is about removing repetitive tasks to free up humans to do the more creative, ad hoc activities. So think of automation as a way of freeing up our brainpower for the things we do best.

For the purposes of this chapter, we mostly ignore manual exploratory testing. This isn't to say that this type of testing isn't important, but just that the scope of this chapter is to focus primarily on how testing microservices differs from testing more typical monolithic applications. But when it comes to automated tests, how many of each test do we want? Another model will help us answer this question and understand what the different trade-offs might be.

Test Scope

In his book *Succeeding with Agile*,[2] Mike Cohn outlines a model called the test pyramid to help explain what types of automated tests are needed. The pyramid helps us think

about not only the scope of the tests, but also the proportions of the different types of tests we should aim for. Cohn's original model split automated tests into unit tests, service tests, and UI tests, as illustrated in Figure 9-2.

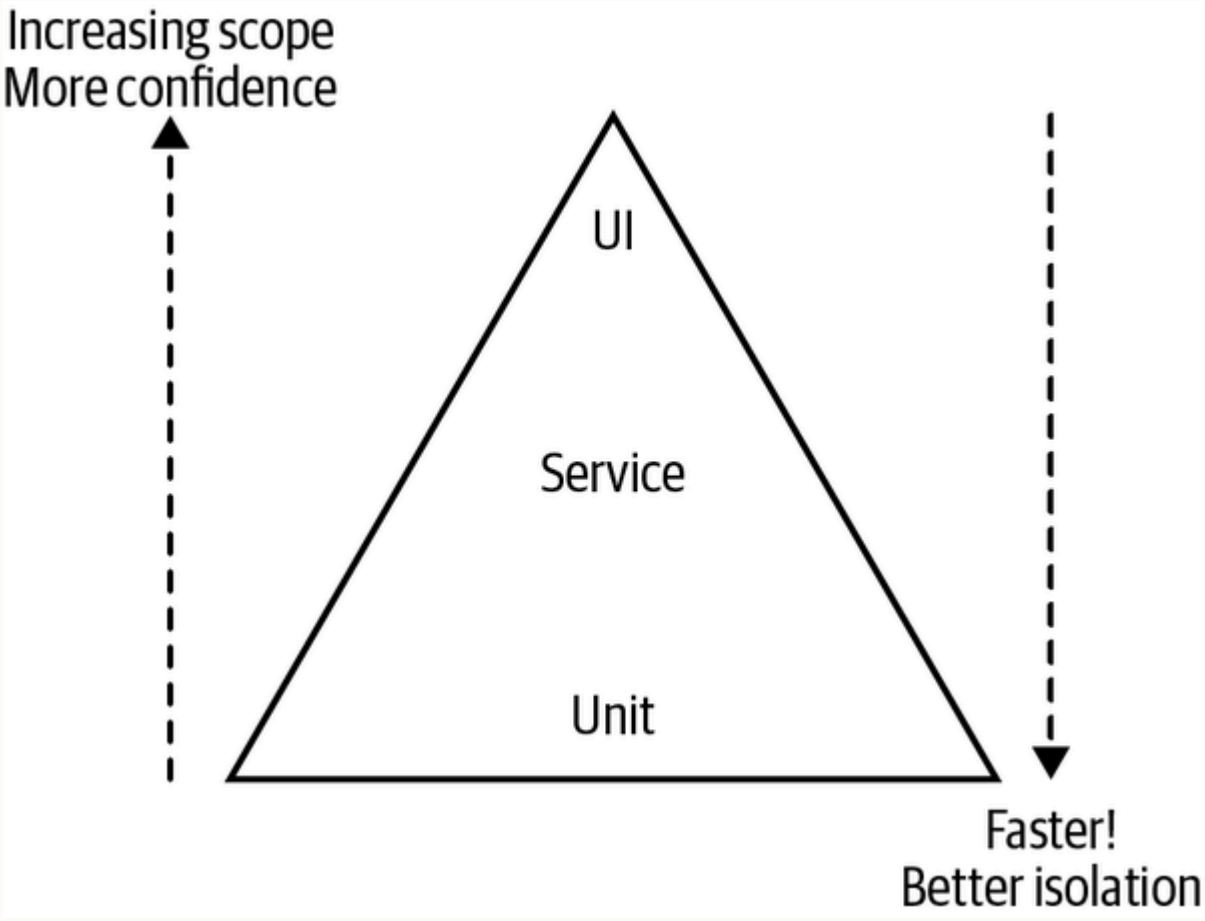


Figure 9-2. Mike Cohn's test pyramid. Mike Cohn, *Succeeding with Agile: Software Development Using Scrum*, 1st ed., © 2010

The key thing to take away when reading the pyramid is that as we go up the pyramid, the test scope increases, as does our confidence that the functionality being tested works. On the other hand, the feedback cycle time increases as the tests take longer to run, and when a test fails, it can be harder to determine which functionality has broken. As we go down the pyramid, in general the tests become much faster, so we get much faster feedback cycles. We find broken functionality faster, our continuous integration builds are faster, and we are less likely to move on to a new task before finding out we have broken something. When those smaller-scoped tests fail, we also tend to know what broke, often down to the exact line of code—each test is better *isolated*, making it easier for us to understand and fix breakages. On the flip side, we don't get a lot of confidence that our system as a whole works if we've tested only one line of code!

The problem with this model is that all these terms mean different things to different people. *Service* is especially overloaded, and there are many definitions of a unit test out there. Is a test a unit test if I test only one line of code? I'd say it is. Is it still a unit test if I test multiple functions or classes? I'd say no, but many would disagree! I tend to stick

with the *unit* and *service* names despite their ambiguity, but much prefer calling *UI* tests *end-to-end* tests, which I'll do from this point forward.

Virtually every team I've worked on has used different names for tests than the ones that Cohn uses in the pyramid. Whatever you call them, the key takeaway is that you will want functional automated tests of different scope for different purposes.

Given the confusion, it's worth looking at what these different layers mean.

Let's look at a worked example. In Figure 9-3, we have our helpdesk application and our main website, both of which are interacting with our `Customer` microservice to retrieve, review, and edit customer details. Our `Customer` microservice is in turn talking to our `Loyalty` microservice, where our customers accrue points by buying Justin Bieber CDs. Probably. This is obviously a sliver of our overall MusicCorp system, but it is a good enough slice for us to dive into a few different scenarios we may want to test.

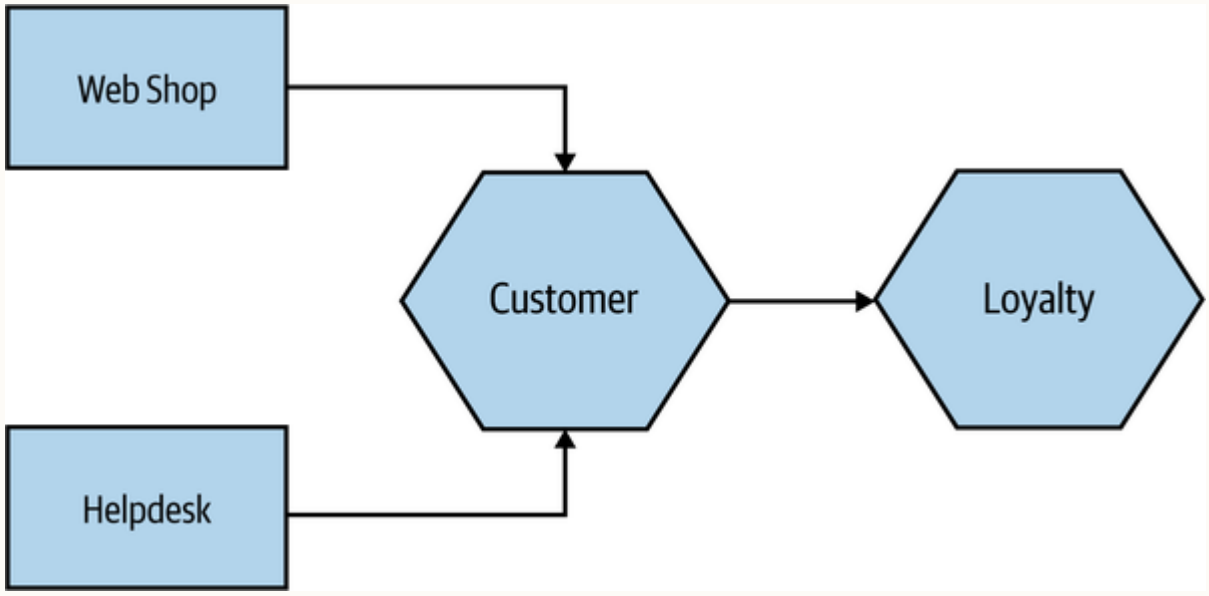


Figure 9-3. Part of our music shop under test

Unit Tests

Unit tests typically test a single function or method call. The tests generated as a side effect of *test-driven design* (TDD) will fall into this category, as do the sorts of tests generated by techniques such as property-based testing. We're not launching microservices here and are limiting the use of external files or network connections. In general, you want a large number of these sorts of tests. Done right, they are very, very fast, and on modern hardware you could expect to run many thousands of these in less than a minute. I know many people who have these tests running automatically when files are changed locally—especially with interpreted languages, this can give very fast feedback cycles.

Unit tests help us developers and thus would be technology facing, not business facing, in Marick's terminology. They are also where we hope to catch most of our bugs. So in our example, when we think about the `Customer` microservice, unit tests would cover small parts of the code in isolation, as shown in Figure 9-4.

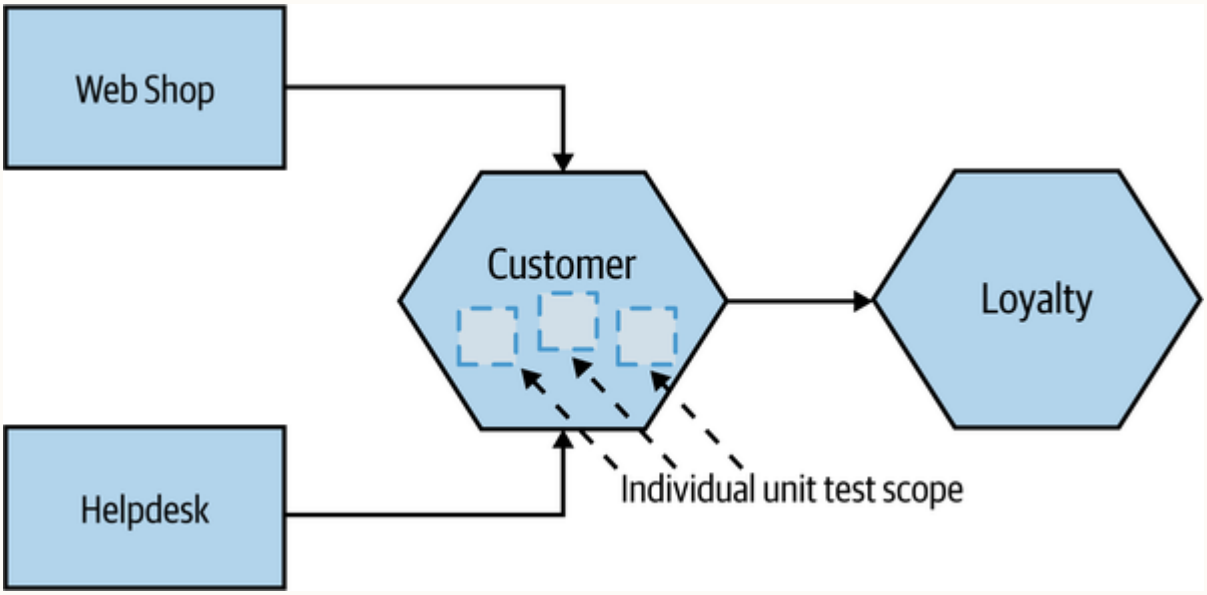


Figure 9-4. Scope of unit tests on our example system

The primary goal of these tests is to give us very fast feedback about whether our functionality is good. Unit tests are also important for supporting refactoring of code, allowing us to restructure our code as we go secure in the knowledge that our small-scoped tests will catch us if we make a mistake.

Service Tests

Service tests are designed to bypass the user interface and test our microservices directly. In a monolithic application, we might just be testing a collection of classes that provide a *service* to the UI. For a system comprising a number of microservices, a service test would test an individual microservice's capabilities.

By running tests against a single microservice in this way, we get increased confidence that the service will behave as we expect, but we still keep the scope of the test somewhat isolated. The cause of the test failure should be limited to just the microservice under test. To achieve this isolation, we need to stub out all external collaborators so only the microservice itself is in scope, as Figure 9-5 shows.

Some of these tests could be as fast as our small-scoped unit tests, but if you decide to test against a real database or to go over networks to stubbed downstream collaborators, test times can increase. These tests also cover more scope than a simple unit test, so when they fail, it can be harder to detect what is broken than with a unit test.

However, they have considerably fewer moving parts and are therefore less brittle than larger-scoped tests.

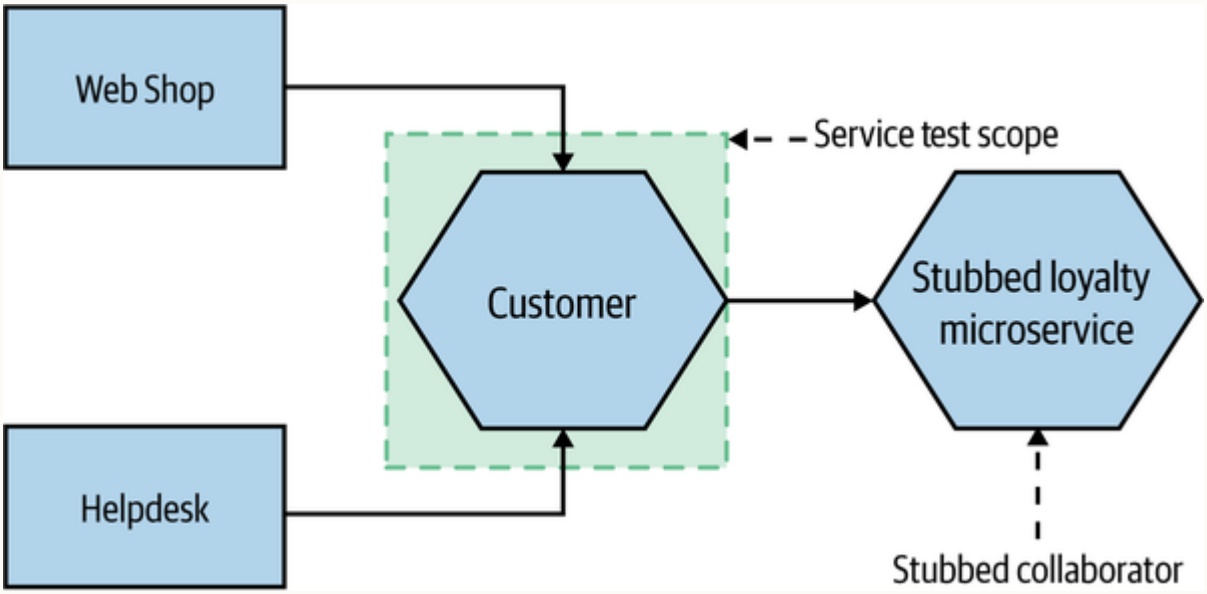


Figure 9-5. Scope of service tests on our example system

End-to-End Tests

End-to-end tests are tests run against your entire system. Often they will be driving a GUI through a browser, but they could easily be mimicking other sorts of user interaction, such as uploading a file.

These tests cover a lot of production code, as we see in Figure 9-6. Thus when they pass, you feel good: you have a high degree of confidence that the code being tested will work in production. But this increased scope comes with downsides, and as we'll see shortly, end-to-end tests can be very tricky to do well in a microservices context.

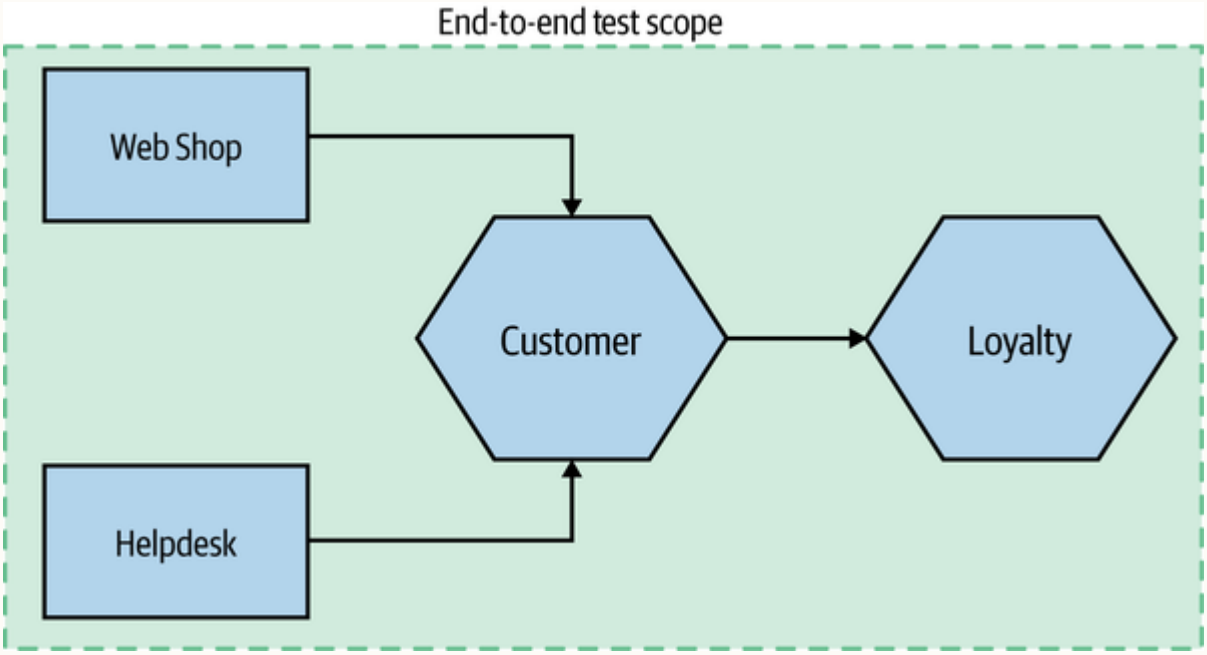


Figure 9-6. Scope of end-to-end tests on our example system

WHAT ABOUT INTEGRATION TESTS?

You may notice that I haven't explicitly described integration tests. This is on purpose. I've found that the term is often used by different people to describe different types of tests. For some, an integration test might only look at the interactions between two services, or perhaps the binding between code and a database. For others, integration tests end up being the same as full end-to-end tests. I've tried to use more explicit terms in this chapter; I hope it will be easy for you to match whatever you refer to as an "integration test" to the terms I use here.

Trade-Offs

What we are striving for with the different types of tests that the pyramid covers is a sensible balance. We want fast feedback, and we want confidence that our system works.

Unit tests are small in scope, so when they fail we can find the problem quickly. They are also quick to write and *really* quick to run. As our tests get larger in scope, we get more confidence in our system, but our feedback starts to suffer as the tests take longer to run. They are also more costly to write and maintain,

You'll frequently be balancing how many of each type of test you need to find that sweet spot. Finding that your test suite is taking too long to run? When broader-scoped tests like our service or end-to-end tests fail, write a smaller-scoped unit test to pick up that breakage sooner. Look to replace some larger-scoped (and slower) tests with faster, smaller-scoped unit tests. On the other hand, when a bug slips through to production, it may be a sign that you're missing a test.

So if these tests all have trade-offs, how many of each test type do you want? A good rule of thumb is that you probably want an order of magnitude more tests as you descend the pyramid. It's important to know that you do have different types of automated tests, and it's important to understand if your current balance gives you a problem!

I worked on one monolithic system, for example, where we had 4,000 unit tests, 1,000 service tests, and 60 end-to-end tests. We decided that from a feedback point of view, we had way too many service and end-to-end tests (the latter of which were the worst offenders in impacting feedback loops), so we worked hard to replace the test coverage with smaller-scoped tests.

A common antipattern is what is often referred to as a *test snow cone*, or inverted pyramid. Here, there are little to no small-scoped tests, with all the coverage in large-scoped tests. These projects often have glacially slow test runs and very long feedback cycles. If these tests are run as part of continuous integration, you won't get many builds, and the nature of the build times means that the build can stay broken for a long period when something does break.

Implementing Service Tests

Implementing unit tests is a fairly simple affair in the grand scheme of things, and there is plenty of documentation out there explaining how to write them. The service and end-to-end tests are the ones that are more interesting, especially in the context of microservices, so that's what we'll focus on next.

Our service tests want to test a slice of functionality across the whole microservice, and only that microservice. So if we wanted to write a service test for the `Customer` microservice from Figure 9-3, we would deploy an instance of the `Customer` microservice—and as discussed earlier, we would want to stub out the `Loyalty` microservice to better ensure that a test breakage can be mapped to an issue with the `Customer` microservice itself.

As we explored in Chapter 7, once we check in our software, one of the first things our automated build will do is create a binary artifact for our microservice—for example, creating a container image for that version of the software. So deploying that is pretty straightforward. But how do we handle faking the downstream collaborators?

Our service test suite needs to stub out downstream collaborators and configure the microservice under test to connect to the stub services. We then need to configure the stubs to send back responses to mimic the real-world microservices.

Mocking or Stubbing

When I talk about stubbing downstream collaborators, I mean that we create a stub microservice that responds with canned responses to known requests from the microservice under test. For example, I might tell my stubbed `Loyalty` microservice that when asked for the balance of customer 123, it should return 15,000. The test doesn't care if the stub is called 0, 1, or 100 times. A variation on this is to use a mock instead of a stub.

When using a mock, I actually go further and make sure the call was made. If the expected call is not made, the test fails. Implementing this approach requires more smarts in the fake collaborators that we create, and if overused it can cause tests to become brittle. As noted, however, a stub doesn't care if it is called 0, 1, or many times.

Sometimes, though, mocks can be very useful to ensure that the expected side effects happen. For example, I might want to check that when I create a customer, a new points balance is set up for that customer. The balance between stubbing and mocking calls is a delicate one and is just as fraught in service tests as in unit tests. In general, though, I use stubs far more than mocks for service tests. For a more in-depth discussion of this trade-off, take a look at *Growing Object-Oriented Software, Guided by Tests* by Steve Freeman and Nat Pryce.[3]

In general, I rarely use mocks for this sort of testing. But having a tool that can implement both mocks and stubs is useful.

While I feel that stubs and mocks are actually fairly well differentiated, I know the distinction can be confusing to some, especially when some people throw in other terms like *fakes*, *spies*, and *dummies*. Gerard Meszaros calls all of these things, including stubs and mocks, "Test Doubles".

A Smarter Stub Service

Normally, for stub services, I've rolled them myself. I've used everything from the Apache web server or nginx to embedded Jetty containers or even command-line-launched Python web servers to launch stub servers for such test cases. I've probably reproduced the same work time and time again in creating these stubs. An old Thoughtworks colleague of mine, Brandon Byars, has potentially saved many of us a chunk of work with his stub/mock server called mountebank.

You can think of mountebank as a small software appliance that is programmable via HTTP. The fact that it happens to be written in NodeJS is completely opaque to any calling service. When mountebank launches, you send it commands telling it to create one or more "imposters," which will respond on a given port with a specific protocol (currently TCP, HTTP, HTTPS, and SMTP are supported), and what responses these imposters should send when requests are sent. It also supports setting expectations if you want to use it as a mock. Because a single mountebank instance can support the creation of multiple imposters, you can use it to stub out multiple downstream microservices.

mountebank does have uses outside of automated functional testing. Capital One made use of mountebank to replace existing mocking infrastructure for their large-scale performance tests, for example.[4]

One limitation of mountebank is that it doesn't support stubbing for messaging protocols—for example, if you wanted to make sure that an event was properly sent (and maybe received) via a broker, you'll have to look elsewhere for a solution. This is one area where Pact might be able to help—that's something we'll look at more shortly.

So if we want to run our service tests for just our `Customer` microservice, we can launch the `Customer` microservice and a mountebank instance that acts as our `Loyalty` microservice on the same machine. And if those tests pass, I can deploy the `Customer` service straightaway! Or can I? What about the services that call the `Customer` microservice—the helpdesk and the web shop? Do we know if we have made a change that may break them? Of course, we have forgotten the important tests at the top of the pyramid: the end-to-end tests.

Implementing (Those Tricky) End-to-End Tests

In a microservice system, the capabilities we expose via our user interfaces are delivered by a number of microservices. The point of the end-to-end tests as outlined in Mike Cohn's pyramid is to drive functionality through these user interfaces against everything underneath to give us some feedback on the quality of the system as a whole.

So, to implement an end-to-end test, we need to deploy multiple microservices together, and then run a test against all of them. Obviously, this test has a much larger scope, resulting in more confidence that our system works! On the other hand, these tests are liable to be slower and make it harder to diagnose failure. Let's dig into them a bit more using our previous example to see how these tests can fit in.

Imagine we want to push out a new version of the `Customer` microservice. We want to deploy our changes into production as soon as possible but are concerned that we may have introduced a change that could break either the helpdesk or the web shop. No problem—let's deploy all our services together and run some tests against the helpdesk and web shop to see if we've introduced a bug. Now, a naive approach would be to just add these tests onto the end of our customer service pipeline, as in Figure 9-7.

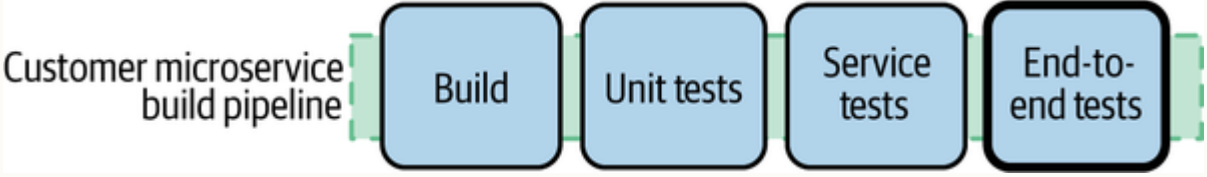


Figure 9-7. Adding our end-to-end tests stage: the right approach?

So far, so good. But the first question we have to ask ourselves is, which version of the other microservices should we use? Should we run our tests against the versions of helpdesk and web shop that are in production? It's a sensible assumption—but what if a new version of either the helpdesk or the web shop is queued up to go live? What should we do then?

Here's another problem: if we have a set of `Customer` end-to-end tests that deploy lots of microservices and run tests against them, what about the end-to-end tests that the other microservices run? If they are testing the same thing, we may find ourselves covering lots of the same ground and may duplicate much of the effort to deploy all those microservices in the first place.

We can deal with both of these problems elegantly by having multiple pipelines "fan-in" to a single end-to-end test stage. Here, when one of a number of different builds is triggered, it can result in shared build steps being triggered. For example, in Figure 9-8, a successful build for any of the four microservices would end up triggering the shared end-to-end tests stage. Some CI tools with better build pipeline support will enable fan-in models like this out of the box.

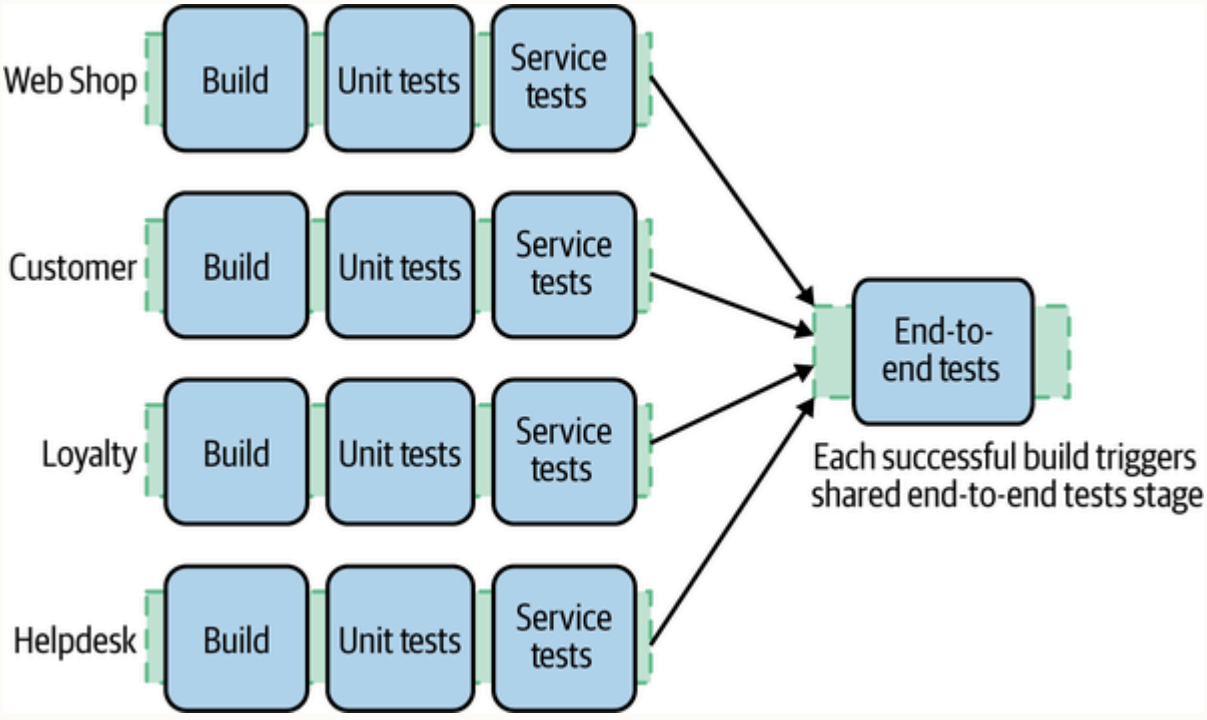


Figure 9-8. A standard way to handle end-to-end tests across services

So any time one of our services changes, we run the tests local to that service. If those tests pass, we trigger our integration tests. Great, eh? Well, there are unfortunately many disadvantages to end-to-end testing.

Flaky and Brittle Tests

As test scope increases, so too does the number of moving parts. These moving parts can introduce test failures that do not show that the functionality under test is broken but indicate that some other problem has occurred. As an example, if we have a test to verify that we can place an order for a single CD and we are running that test against four or five microservices, if any of them is down, we could get a failure that has nothing to do with the nature of the test itself. Likewise, a temporary network glitch could cause a test to fail without saying anything about the functionality under test.

The more moving parts there are, the more brittle our tests may be and the less deterministic they are. If you have tests that *sometimes* fail, but everyone just reruns them because they may pass again later, then you have flaky tests. And tests covering lots of different processes are not the only culprit. Tests that cover functionality being exercised on multiple threads (and across multiple processes) are also often problematic; a failure could mean a race condition or a time-out, or that the functionality is actually broken. Flaky tests are the enemy. When they fail, they don't tell us much. We rerun our CI builds in the hope that they will pass again later, only to see check-ins pile up, and suddenly we find ourselves with a load of broken functionality.

When we detect flaky tests, it is essential that we do our best to remove them. Otherwise, we start to lose faith in a test suite that "always fails like that." A test suite with flaky tests can become a victim of what Diane Vaughan calls the *normalization of deviance*—the idea that over time we can become so accustomed to things being wrong that we start to accept them as being normal and not a problem.[5] This very human tendency means we need to find and eliminate these flaky tests as soon as we can before we start to assume that failing tests are OK.

In "Eradicating Non-Determinism in Tests,"[6] Martin Fowler advocates the approach that if you have flaky tests, you should track them down and—if you can't immediately fix them—remove them from the suite so you can treat them. See if you can rewrite them to avoid testing code running on multiple threads. See if you can make the underlying environment more stable. Better yet, see if you can replace the flaky test with a smaller-scoped test that is less likely to exhibit problems. In some cases, changing the software under test to make it easier to test can also be the right way forward.

Who Writes These End-to-End Tests?

With the tests that run as part of the pipeline for a specific microservice, the sensible starting point is for the team that owns that service to write those tests (we'll talk more about service ownership in Chapter 15). But if we consider that we might have multiple teams involved, and the end-to-end tests step is now effectively shared among the teams, who writes and looks after these tests?

I have seen a number of problems caused here. These tests become a free-for-all, with all teams granted access to add tests without any understanding of the health of the whole suite. This can often result in an explosion of test cases, sometimes culminating in the test snow cone we talked about earlier. I have also seen situations in which, because there was no real obvious ownership of these tests, their results get ignored. When they break, everyone assumes it is someone else's problem, so they don't care whether the tests are passing.

One solution I've seen here is to designate certain end-to-end tests as being the responsibility of a given team, even though they might cut across microservices being worked on by multiple different teams. I first learned of this approach from Emily Bache.[7] The idea is that even though we make use of a "fan in" stage in our pipeline, they would then split the end-to-end test suite into groups of functionality that were owned by different teams, as we'll see in Figure 9-9.

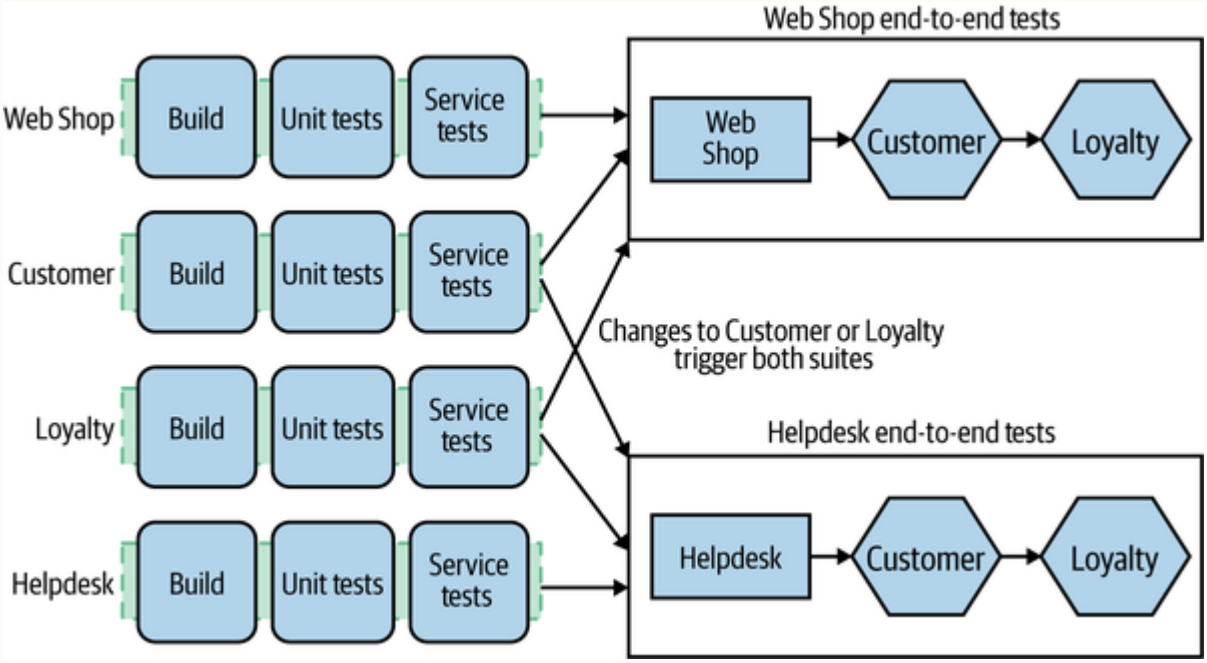


Figure 9-9. A standard way to handle end-to-end tests across services

In this particular example, a change to the `Web Shop` that passes the service test stage would trigger the associated end-to-end tests, with the suite being owned by the same team that owns the `Web Shop`. Likewise, any changes to the `Helpdesk` only will trigger

the associated end-to-end tests. But changes to either `Customer` or `Loyalty` trigger *both* sets of tests. This can lead us to a situation in which a change made to the `Loyalty` microservice could break both sets of end-to-end tests, potentially requiring the teams who own these two test suites to chase the owner of the `Loyalty` microservice for a fix. Although this model helped in Emily's case, as we can see, it still has its challenges. Fundamentally, it's problematic to have a team own responsibility for tests where people from a different team can cause these tests to break.

Sometimes organizations react by having a dedicated team write these tests. This can be disastrous. The team developing the software becomes increasingly distant from the tests for its code. Cycle times increase, as service owners end up waiting for the test team to write end-to-end tests for the functionality it just wrote. Because another team writes these tests, the team that wrote the service is less involved with, and therefore less likely to know, how to run and fix these tests. Although it is unfortunately still a common organizational pattern, I see significant harm done whenever a team is distanced from writing tests for the code it wrote in the first place.

Getting this aspect right is really hard. We don't want to duplicate effort, nor do we want to completely centralize this to the extent that the teams building services are too far removed from things. If you can find a clean way to assign end-to-end tests to a specific team, then do it. If not, and if you can't find a way to remove end-to-end tests and replace them with something else, then you'll likely need to treat the end-to-end test suite as a shared codebase, but with joint ownership. Teams are free to check in to this suite, but ownership of the health of the suite has to be shared between the teams developing the services themselves. If you want to make extensive use of end-to-end tests with multiple teams, I think this approach is essential, and yet I have seen it done only rarely, and never without issues. Ultimately, I am convinced that at a certain level of organizational scale, you need to move away from cross-team end-to-end tests for this reason.

How Long Should End-to-End Tests Run?

These end-to-end tests can take a while. I have seen them take up to a day to run, if not longer, and on one project I worked on, a full regression suite took six weeks! I rarely see teams actually curate their end-to-end test suites to reduce overlap in test coverage, or spend enough time in making them fast.

This slowness, combined with the fact that these tests can often be flaky, can be a major problem. A test suite that takes all day and often has breakages that have nothing to do with broken functionality is a disaster. Even if your functionality *is* broken, it could take

you many hours to find out—at which point you likely would have already moved on to other activities, and the context switch in shifting your brain back to fix the issue would be painful.

We can ameliorate some of this by running tests in parallel—for example, making use of tools like Selenium Grid. However, this approach is not a substitute for actually understanding what needs to be tested and actively *removing* tests that are no longer needed.

Removing tests is sometimes a fraught exercise, and I suspect those who attempt it have much in common with people who want to remove certain airport security measures. No matter how ineffective the security measures might be, any conversation about removing them is often countered with knee-jerk reactions about not caring about people's safety or wanting terrorists to win. It is hard to have a balanced conversation about the value something adds versus the burden it entails. It can also be a difficult risk/reward trade-off. Do you get thanked if you remove a test? Maybe. But you'll certainly get blamed if a test you removed lets a bug through. When it comes to the larger-scoped test suites, however, this is exactly what we need to be able to do. If the same feature is covered in 20 different tests, perhaps we can get rid of half of them, as those 20 tests take 10 minutes to run! What this requires is a better understanding of risk, which is something humans are famously bad at. As a result, this intelligent curation and management of larger-scoped, high-burden tests happens incredibly infrequently. Wishing people did this more isn't the same thing as making it happen.

The Great Pile-Up

The long feedback cycles associated with end-to-end tests aren't just a problem when it comes to developer productivity. With a long test suite, any breaks take a while to fix, which reduces the amount of time that the end-to-end tests can be expected to be passing. If we deploy only software that has passed through all our tests successfully (which we should!), it means fewer of our services get through to the point of being deployable into production.

This can lead to a pile-up. While a broken integration test stage is being fixed, more changes from upstream teams can pile in. Aside from the fact that this can make fixing the build harder, it means the scope of changes to be deployed increases. The ideal way to handle this is to not let people check in if the end-to-end tests are failing, but given a long test suite time, that is often impractical. Try saying, "You 30 developers: no check-ins until we fix this seven-hour-long build!" Allowing check-ins on a broken end-to-end test suite, though, is really fixing the wrong problem. If you allow check-ins on a broken build, the build may stay broken for longer, undermining its effectiveness as a way to

give you fast feedback about the quality of the code. The right answer is to make the test suite faster.

The larger the scope of a deployment and the higher the risk of a release, the more likely we are to break something. So we want to make sure we can release small, well-tested changes frequently. When end-to-end tests slow down our ability to release small changes, they can end up doing more harm than good.

The Metaversion

With the end-to-end tests step, it is easy to start thinking, *I know all these services at these versions work together, so why not deploy them all together?* This very quickly becomes a conversation along the lines of, *So why not use a version number for the whole system?* To quote Brandon Byars, "Now you have 2.1.0 problems."

By versioning together changes made to multiple services, we effectively embrace the idea that changing and deploying multiple services at once is acceptable. It becomes the norm; it becomes OK. In doing so, we cede one of the main advantages of a microservice architecture: the ability to deploy one service by itself, independently of other services.

All too often, the approach of accepting multiple services being deployed together drifts into a situation in which services become coupled. Before long, nicely separate services become increasingly tangled with others, and you never notice, because you never try to deploy them by themselves. You end up with a tangled mess where you have to orchestrate the deployment of multiple services at once, and as we discussed previously, this sort of coupling can leave us in a worse place than we would be with a single monolithic application.

This is bad.

Lack of Independent Testability

We've come back frequently to the topic of independent deployability being an important property to facilitate teams working in a more autonomous way, allowing for software to be shipped more effectively. If your teams work independently, it follows that they should be able to test independently. As we've seen, end-to-end tests can reduce the autonomy of teams and can force increased levels of coordination, with the associated challenges that can bring.

The drive toward independent testability extends to our use of the infrastructure associated with testing. Often, I see people having to make use of shared testing environments in which tests from multiple teams are executed. Such an environment is often highly constrained, and any issue can cause significant problems. Ideally, if you want your teams to be able to develop and test in an independent fashion, they should have their own test environments too.

The research summarized in *Accelerate* found that high-performing teams were more likely to "do most of their testing on demand, without requiring an integrated test environment."[8]

Should You Avoid End-to-End Tests?

Despite the disadvantages just outlined, for many users end-to-end tests can still be manageable with a small number of microservices, and in these situations they still make a lot of sense. But what happens with 3, 4, 10, or 20 services? Very quickly these test suites become hugely bloated, and in the worst case they can result in a Cartesian-like explosion in the scenarios under test.

In fact, even with a small number of microservices, these tests become difficult when you have multiple teams sharing end-to-end tests. With a shared end-to-end test suite, you undermine your goal of independent deployability. Your ability as a team to deploy a microservice now requires that a test suite shared by multiple teams passes.

What is one of the key problems we are trying to address when we use the end-to-end tests outlined previously? We are trying to ensure that when we deploy a new service to production, our changes won't break consumers. Now, as we covered in detail in "Structural Versus Semantic Contract Breakages", having explicit schemas for our microservice interfaces can help us catch structural breakages, and that can definitely reduce the need for more complex end-to-end tests.

However, schemas can't pick up semantic breakages, namely changes in *behavior* that cause breakages due to backward incompatibility. End-to-end tests absolutely can help catch these semantic breakages, but they do so at a great cost. Ideally, we'd want to have some type of test that can pick up semantic breaking changes and run in a reduced scope, improving test isolation (and therefore speed of feedback). This is where contract tests and consumer-driven contracts come in.

Contract Tests and Consumer-Driven Contracts (CDCs)

With *contract tests*, a team whose microservice consumes an external service writes tests that describe how it expects an external service will behave. This is less about testing your own microservice and more about specifying how you expect an external service to behave. One of the main reasons that these contract tests can be useful is that they can be run against any stubs or mocks you are using that represent external services—your contract tests should pass when you run your own stubs, just as they should the real external service.

Contract tests become very useful when used as part of *consumer-driven contracts* (CDCs). The contract tests are in effect an explicit, programmatic representation of how the consumer (upstream) microservice expects the producer (downstream) microservice to behave. With CDCs, the consumer team ensures that these contract tests are shared with the producer team to allow the producer team to ensure that its microservice meets these expectations. Typically, this is done by having the downstream producer team run the consumer contracts for each consuming microservice as part of its test suite that would be run on every build. Very importantly from a test feedback point of view, these tests need to be run only against a single producer in isolation, so they can be faster and more reliable than the end-to-end tests they might replace.

As an example, let's revisit our scenario from earlier. The `Customer` microservice has two separate consumers: the helpdesk and the web shop. Both of these consuming applications have expectations for how the `Customer` microservice will behave. In this example, you create a set of tests for each consumer: one representing the helpdesk's expectations of the `Customer` microservice, and another set representing the expectations that the web shop has.

Because these CDCs are expectations for how the `Customer` microservice should behave, we need only to run the `Customer` microservice itself, meaning we have the same effective test scope as our service tests. They'd have similar performance characteristics and would require us to run only the `Customer` microservice itself, with any external dependencies stubbed out.

A good practice here is to have someone from the producer and consumer teams collaborate on creating the tests, so perhaps people from the web shop and helpdesk teams pair with people from the customer service team. Arguably, consumer-driven contracts are as much about fostering clear lines of communication and collaboration, where needed, between microservices and the teams that consume them. It could be argued, in fact, that implementing CDCs is just making more explicit the communication between

the teams that must already exist. In cross-team collaboration, CDCs are an explicit reminder of Conway's law.

CDCs sit at the same level in the test pyramid as service tests, albeit with a very different focus, as shown in Figure 9-10. These tests are focused on how a consumer will use the service, and the trigger if they break is very different when compared with service tests. If one of these CDCs breaks during a build of the `Customer` service, it becomes obvious which consumer would be impacted. At this point, you can either fix the problem or else start the discussion about introducing a breaking change in the manner we discussed in "Handling Change Between Microservices". So with CDCs, we can identify a breaking change prior to our software going into production without having to use a potentially expensive end-to-end test.

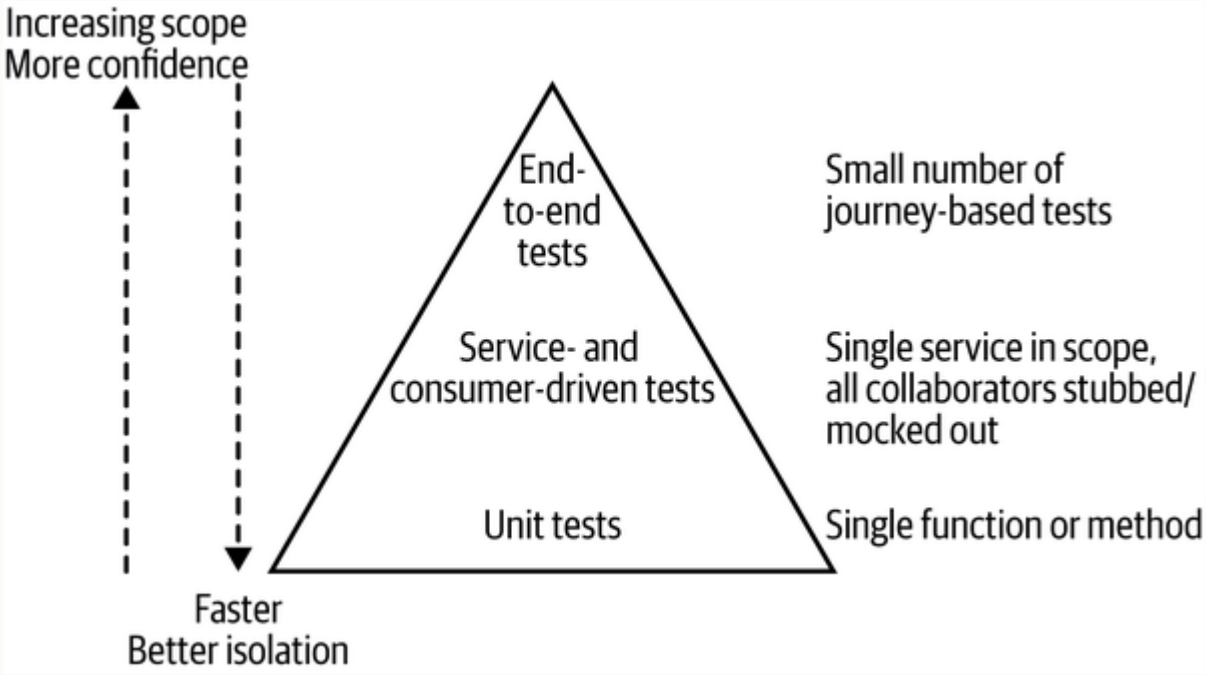


Figure 9-10. Integrating consumer-driven tests into the test pyramid

Pact

Pact is a consumer-driven testing tool that was originally developed in-house at realestate.com.au but is now open source. Originally just for Ruby and focused only on HTTP protocols, Pact now supports multiple languages and platforms, such as the JVM, JavaScript, Python, and .NET, and can also be used with messaging interactions.

With Pact, you start by defining the expectations of the producer using a DSL in one of the supported languages. Then you launch a local Pact server and run this expectation against it to create the Pact specification file. The Pact file is just a formal JSON specification; you could obviously handcode this, but using the language-specific SDK is much easier.

A really nice property of this model is that the locally running mock server used to generate the Pact file also works as a local stub for downstream microservices. By defining your expectations locally, you are defining how this local stub service should respond. This can replace the need for tools like mountebank (or your own hand-rolled stubbing or mocking solutions).

On the producer side, you then verify that this consumer specification is met by using the JSON Pact specification to drive calls against your microservice and verify responses. For this to work, the producer needs access to the Pact file. As we discussed earlier in "Mapping Source Code and Builds to Microservices", we expect the consumer and producer to be in different builds. This means that we need some way for this JSON file, which will be generated to the consumer build, to be made available by the producer.

You could store the Pact file in your CI/CD tool's artifact repository, or else you could use the Pact Broker, which allows you to store multiple versions of your Pact specifications. This could let you run your consumer-driven contract tests against multiple different versions of the consumers, if you wanted to test against, say, the version of the consumer in production and the version of the consumer that was most recently built.

The Pact Broker actually has a host of useful capabilities. Aside from acting as a place where contracts can be stored, you can also find out when those contracts were validated. Also, because the Pact Broker knows about the relationship between the consumer and producer, it's able to show you which microservices depend on which other microservices.

Other options

Pact isn't the only option for tooling around consumer-driven contracts. Spring Cloud Contract is one such example. However, it's worth noting that—unlike Pact, which was designed from the beginning to support different technology stacks—Spring Cloud Contract is really only useful in a pure JVM ecosystem.

It's about conversations

In Agile, stories are often referred to as a placeholder for a conversation. CDCs are just like that. They become the codification of a set of discussions about what a service API should look like, and when they break, they become a trigger point to have conversations about how that API should evolve.

It is important to understand that CDCs require good communication and trust between the consumer and producing service. If both parties are in the same team (or are the same person!), then this shouldn't be hard. However, if you are consuming a service provided with a third party, you may not have the frequency of communication, or trust, to make CDCs work. In these situations, you may have to make do with limited larger-scoped integration tests just around the *untrusted* component. Alternatively, if you are creating an API for thousands of potential consumers, such as with a publicly available web service API, you may have to play the role of the consumer yourself (or perhaps work with a subset of your consumers) in defining these tests. Breaking huge numbers of external consumers is a pretty bad idea, so if anything the importance of CDCs is increased!

The Final Word

As outlined in detail earlier in the chapter, end-to-end tests have a large number of disadvantages that grow significantly as you add more moving parts under test. From speaking to people who have been implementing microservices at scale for a while now, I have learned that most of them over time remove the need entirely for end-to-end tests in favor of other mechanisms to validate the quality of their software—for example, the use of explicit schemas and CDCs, in-production testing, or perhaps some of the progressive delivery techniques we've discussed, such as canary releases.

You can view the running of end-to-end tests prior to production deployment as training wheels. While you are learning how CDCs work and improving your production monitoring and deployment techniques, these end-to-end tests may form a useful safety net, where you are trading off cycle time for decreased risk. But as you improve those other areas, and as the relative cost of creating the end-to-end tests increases, you can start to reduce your reliance on end-to-end tests to the point that they are no longer needed. Ditching end-to-end tests without fully understanding what you've lost is probably a bad idea.

Obviously you'll have a better understanding of your own organization's risk profile than I, but I would challenge you to think long and hard about how much end-to-end testing you really need to do.

Developer Experience

One of the significant challenges that can emerge as developers find the need to work on more and more microservices is that the developer experience can start to suffer, for

the simple reason that they are trying to run more and more microservices locally. This often arises in situations in which a developer needs to run a large-scoped test that bridges multiple nonstubbed microservices.

How quickly this becomes an issue will depend on a number of factors. How many microservices a developer needs to run locally, which technology stack those microservices are written in, and the power of the local machine can all play a part. Some technology stacks are more resource intensive in terms of their initial footprint—JVM-based microservices come to mind. On the other hand, some tech stacks can result in microservices with a faster and more lightweight resource footprint, perhaps allowing you to run many more microservices locally.

One approach to dealing with this challenge is to instead have developers do their development and test work in a cloud environment. The idea is that you can have many more resources available to you to run the microservices you need. Aside from the fact that this model requires that you always have connectivity to your cloud resources, the other main issue is that your feedback cycles can suffer. If you need to make a code change locally and upload the new version of this code (or a locally built artifact) to the cloud, it can add significant delay to your development and test cycles, especially if you are operating in a part of the world with more constrained internet connectivity.

Full in-the-cloud development is one possibility for addressing the issue of feedback cycles; cloud-based IDEs like Cloud9, now owned by AWS, have shown that this is possible. However, while something like this might be the future for development, it's certainly not the present for the vast majority of us.

Fundamentally, I do think that the use of cloud environments to allow a developer to run more microservices for their development and test cycles is missing the point, resulting in more complexity than is needed, in addition to higher costs. Ideally, you want to aim for a developer needing to run only the microservices they actually work on. If a developer is part of a team that owns five microservices, then that developer needs to be able to run those microservices as effectively as possible, and for fast feedback my preference would always be for them to run locally.

But what if the five microservices your team owns want to call out to other systems and microservices that are owned by other teams? Without them, the local development and test environment won't function, will it? Here again, stubbing comes to the rescue. I should be able to stand up local stubs that mimic the microservices that are out of scope for my team. The only real microservices you should be running locally are the ones you are working on. If you are working in an organization where you are expected to work on

hundreds of different microservices, well then, you've got much bigger problems to deal with—this is a topic we'll explore in more depth in "Strong Versus Collective Ownership".

From Preproduction to In-Production Testing

Historically, most of the focus of testing has been around testing our systems *before* we get to production. With our tests, we are defining a series of models with which we hope to prove whether our system works and behaves as we would like, both functionally and nonfunctionally. But if our models are not perfect, we will encounter problems when our systems are used in anger. Bugs slip into production, new failure modes are discovered, and our users use the system in ways we could never expect.

One reaction to this is often to define more and more tests, and refine our models, to catch more issues early and reduce the number of problems we encounter with our running production system. However, at a certain point we have to accept that we hit diminishing returns with this approach. With testing prior to deployment, we cannot reduce the chance of failure to zero.

The complexity of a distributed system is such that it can be infeasible to catch all potential problems that might occur before we hit production itself.

Generically speaking, the purpose of a test is to give us feedback as to whether or not our software is of sufficient quality. Ideally, we want that feedback as soon as possible, and we'd like to be able to spot if there is a problem with our software before an end user experiences that issue. This is why a lot of testing is done before we release our software.

Limiting ourselves to testing only in a preproduction environment, though, is hamstringing us. We are reducing the places in which we can pick up issues, and we're also eliminating the possibility of testing the quality of our software in the most important location—where it is going to be used.

We can and should also look to apply testing in a production environment. This can be done in a safe manner, can provide higher-quality feedback than preproduction testing, and, as we see, is likely something you're already doing, whether you realize it or not.

Types of In-Production Testing

There is a long list of different tests that we can carry out in production, ranging from the simple to the complex. To start off with, let's think about something as simple as a ping check to make sure that a microservice is live. Simply checking to see if a microservice instance is running is a type of test—we just don't see it as that, as it is an activity that is typically handled by "operations" people. But fundamentally, something as simple as determining whether or not a microservice is up can be viewed as a test—one that we run frequently on our software.

Smoke tests are another example of in-production testing. Typically done as part of the deployment activities, a smoke test ensures that the deployed software is operating correctly. These smoke tests will normally be done on the real, running software before it is released to the users (more on that shortly).

Canary releases, which we covered in Chapter 8, are also a mechanism that is arguably about testing. We release a new version of our software to a small portion of users to "test" that it works correctly. If it does, we can roll out the software to a larger part of our user base, perhaps in a fully automated way.

Another example of in-production testing is injecting fake user behavior into the system to make sure it works as expected—for example, placing an order for a fake customer, or registering a new (fake) user in the real production system. This type of test can sometimes get pushback, because people are worried about the impact it might have on the production system. So if you create tests like this, be sure to make them safe.

Making Testing in Production Safe

If you decide to do testing in production (and you should!), it's important that the tests not cause production issues, either by introducing system instability or by tainting production data. Something as simple as pinging a microservice instance to make sure it is live is likely to be a safe operation—if this causes system instability, you likely have pretty serious issues that need addressing, unless you've accidentally made your health check system an internal denial of service attack.

Smoke tests are typically safe, as the operations they carry out are often done on software before it is released. As we explored in "Separating Deployment from Release", separating the concept of deployment from release can be incredibly useful. When it comes to in-production testing, tests carried out on software that is deployed into production, before it is released, should be safe.

People tend to get most concerned about the safety of things like injecting fake user behavior into the system. We don't really want the order to be shipped, or the payment made. This is something that needs due care and attention, and despite the challenges, this type of testing can be hugely beneficial. We'll return to this in "Semantic Monitoring".

Mean Time to Repair over Mean Time Between Failures?

So by looking at techniques like blue-green deployment or canary releasing, we find a way to test closer to (or even in) production, and we also build tools to help us manage a failure if it occurs. Using these approaches is a tacit acknowledgment that we cannot spot and catch all problems before we actually release our software.

Sometimes expending the same effort on getting better at fixing problems when they occur can be significantly more beneficial than adding more automated functional tests. In the web operations world, this is often referred to as the trade-off between optimizing for *mean time between failures (MTBF)* and optimizing for *mean time to repair (MTTR)*.

Techniques to reduce the time to recovery can be as simple as very fast rollbacks coupled with good monitoring (which we'll discuss in Chapter 10). If we can spot a problem in production early and roll back early, we reduce the impact on our customers.

For different organizations, the trade-off between MTBF and MTTR will vary, and much of this lies with understanding the true impact of failure in a production environment. However, most organizations that I see spending time creating functional test suites often expend little to no effort on better monitoring or recovering from failure. So while they may reduce the number of defects that occur in the first place, they can't eliminate all of them, and are unprepared for dealing with them if they pop up in production.

Trade-offs other than those between MTBF and MTTR exist. For example, whether you are trying to work out if anyone will actually use your software, it may make much more sense to get something out now, to prove the idea or the business model before building robust software. In an environment where this is the case, testing may be overkill, as the impact of not knowing if your idea works is much higher than having a defect in production. In these situations, it can be quite sensible to avoid testing prior to production altogether.

Cross-Functional Testing

The bulk of this chapter has been focused on testing specific pieces of functionality, and how this differs when you are testing a microservice-based system. However, there is another category of testing that is important to discuss. *Nonfunctional requirements* is an umbrella term used to describe those characteristics your system exhibits that cannot simply be implemented like a normal feature. They include aspects like the acceptable latency of a web page, the number of users a system should support, how accessible your user interface should be to people with disabilities, or how secure your customer data should be.

The term *nonfunctional* never sat well with me. Some of the things that get covered by this term seem very functional in nature! A previous colleague of mine, Sarah Taraporewalla, coined the phrase *cross-functional requirements* (CFR) instead, which I greatly prefer. It speaks more to the fact that these system behaviors really only emerge as the result of lots of cross-cutting work.

Many if not most CFRs can really only be met in production. That said, we can define test strategies to help us see if we are at least moving toward meeting these goals. These sorts of tests fall into the Property Testing quadrant. A great example of this type of test is the performance test, which we'll discuss in more depth shortly.

You may want to track some CFRs at an individual microservice level. For example, you may decide that the durability of service you require from your payment service is significantly higher, but you are happy with more downtime for your music recommendation service, knowing that your core business can survive if you are unable to recommend artists similar to Metallica for 10 minutes or so. These trade-offs will end up having a large impact on how you design and evolve your system, and once again the fine-grained nature of a microservice-based system gives you many more chances to make these trade-offs. When looking at the CFRs a given microservice or team might have to take responsibility for, it's common for them to surface as part of a team's service-level objectives (SLOs), a topic we explore further in "Are We Doing OK?".

Tests around CFRs should follow the pyramid too. Some tests will have to be end-to-end, like load tests, but others won't. For example, once you've found a performance bottleneck in an end-to-end load test, write a smaller-scoped test to help you catch the problem in the future. Other CFRs fit faster tests quite easily. I remember working on a project for which we had insisted on ensuring that our HTML markup was using proper accessibility features to help people with disabilities use our website. Checking the gen-

erated markup to make sure that the appropriate controls were there could be done very quickly without the need for any networking round trips.

All too often, considerations about CFRs come far too late. I strongly suggest looking at your CFRs as early as possible and reviewing them regularly.

Performance Tests

Performance tests are worth calling out explicitly as a way of ensuring that some of our cross-functional requirements can be met. When decomposing systems into smaller microservices, we increase the number of calls that will be made across network boundaries. Where previously an operation might have involved one database call, it may now involve three or four calls across network boundaries to other services, with a matching number of database calls. All of this can decrease the speed at which our systems operate. Tracking down sources of latency is especially important. When you have a call chain of multiple synchronous calls, if any part of the chain starts acting slowly, everything is affected, potentially leading to a significant impact. This makes having some way to performance test your applications even more important than it might be with a more monolithic system. Often the reason this sort of testing gets delayed is because initially there isn't enough of the system there to test. I understand this problem, but all too often it leads to kicking the can down the road, with performance testing often only being done just before going live for the first time, if at all! Don't fall into this trap.

As with functional tests, you may want a mix. You may decide that you want performance tests that isolate individual services, but start with tests that check core journeys in your system. You may be able to take end-to-end journey tests and simply run these at volume.

To generate worthwhile results, you'll often need to run given scenarios with gradually increasing numbers of simulated customers. This allows you to see how latency of calls varies with increasing load. This means that performance tests can take a while to run. In addition, you'll want the system to match production as closely as possible, to ensure that the results you see will be indicative of the performance you can expect on the production systems. This can mean that you'll need to acquire a more production-like volume of data and may need more machines to match the infrastructure—tasks that can be challenging. Even if you struggle to make the performance environment truly production-like, the tests may still have value in tracking down bottlenecks. Just be aware that you may get false negatives—or, even worse, false positives.

Due to the time it takes to run performance tests, it isn't always feasible to run them on every check-in. It is a common practice to run a subset every day, and a larger set every week. Whatever approach you pick, make sure you run tests as regularly as you can. The longer you go without running performance tests, the harder it can be to track down the culprit. Performance problems are especially difficult to resolve, so if you can reduce the number of commits you need to look at to see a newly introduced problem, your life will be much easier.

And make sure you also look at the results! I've been very surprised by the number of teams I have encountered who have put in a lot of work implementing tests and running them but never actually check the numbers. Often this is because people don't know what a "good" result looks like. You really need to have targets. When you provide a microservice to be used as part of a wider architecture, it's common to have specific expectations that you commit to deliver—the SLOs I mentioned earlier. If as part of this you commit to deliver a certain level of performance, then it makes sense for any automated test to give you feedback as to whether or not you're likely to meet (and hopefully exceed) that target.

In lieu of specific performance targets, automated performance tests can still be very useful in helping you see how the performance of your microservice varies as you make changes. It can be a safety net to catch you if you make a change that causes a drastic degradation in performance. So an alternative to a specific target might be to fail the test if the delta in performance from one build to the next varies too much.

Performance testing needs to be done in concert with understanding the real system performance (which we'll discuss more in Chapter 10), and ideally you would use the same tools in your performance test environment for visualizing system behavior as those you use in production. This approach can make it much easier to compare like with like.

Robustness Tests

A microservice architecture is often only as reliable as its weakest link, and as a result it's common for our microservices to build in mechanisms to allow them to improve their robustness in order to improve system reliability. We'll explore this topic more in "Stability Patterns", but examples include running multiple instances of a microservice behind a load balancer to tolerate the failure of an instance, or using circuit breakers to programmatically handle situations in which downstream microservices cannot be contacted.

In such situations it can be useful to have tests that allow you to re-create certain failures to ensure that your microservice keeps operating as a whole. By their nature these tests can be a bit more tricky to implement. For example, you might need to create an artificial network time-out between a microservice under test and an external stub. That said, they can be worthwhile, especially if you are creating shared functionality that will be used across multiple microservices—for example, using a default service mesh implementation to handle circuit breaking.

Summary

Bringing this all together, what I have outlined in this chapter is a holistic approach to testing that hopefully gives you some general guidance on how to proceed when testing your own systems. To reiterate the basics:

- Optimize for fast feedback, and separate types of tests accordingly.
- Avoid the need for end-to-end tests that span more than one team—consider using consumer-driven contracts instead.
- Use consumer-driven contracts to provide focus points for conversations between teams.
- Try to understand the trade-off between putting more effort into testing and detecting issues faster in production (optimizing for MTBF versus optimizing for MTTR).
- Give testing in production a try!

If you are interested in reading more about testing, I recommend *Agile Testing* by Lisa Crispin and Janet Gregory (Addison-Wesley), which among other things covers the use of the testing quadrant in more detail. For a deeper dive into the test pyramid, along with some code samples and more tool references, I also recommend "The Practical Test Pyramid" by Ham Vocke.[9]

This chapter focused mostly on making sure our code works before it hits production, but we also started to look at testing our application once it reaches production. This is something we need to explore in a lot more detail. It turns out that microservices cause a multitude of challenges for understanding how our software is behaving in production—a topic we'll cover in more depth next.

1 Lisa Crispin and Janet Gregory, *Agile Testing: A Practical Guide for Testers and Agile Teams* (Upper Saddle River, NJ: Addison-Wesley, 2008).

2 Mike Cohn, *Succeeding with Agile* (Upper Saddle River, NJ: Addison-Wesley, 2009).

3 Steve Freeman and Nat Pryce, *Growing Object-Oriented Software, Guided by Tests* (Upper Saddle River, NJ: Addison-Wesley, 2009).

4 Jason D. Valentino, "Moving One of Capital One's Largest Customer-Facing Apps to AWS," Capital One Tech, May 24, 2017, *https://oreil.ly/5UM5W*.

5 Diane Vaughan, *The Challenger Launch Decision: Risky Technology, Culture, and Deviance at NASA* (Chicago: University of Chicago Press, 1996).

6 Martin Fowler, "Eradicating Non-Determinism in Tests," martinfowler.com, April 14, 2011, *https://oreil.ly/7Ve7e*.

7 Emily Bache, "End-to-End Automated Testing in a Microservices Architecture—Emily Bache," NDC Conferences, July 5, 2017, YouTube video, 56:48, NDC Oslo 2017, *https://oreil.ly/QX3EK*.

8 Nicole Forsgren, Jez Humble, and Gene Kim, *Accelerate: The Science of Building and Scaling High Performing Technology Organizations* (Portland, OR: IT Revolution, 2018).

9 Ham Vocke, "The Practical Test Pyramid," martinfowler.com, February 26, 2018, *https://oreil.ly/J7lc6*.

Previous chapter
‹ 8. Deployment

Next chapter
10. From Monitoring to Observability ›